

Pythonia

Manual de estudio: tronco común

Arquitecto

Nivel 3: Avanzado

Python idiomático, eficiente y listo para producción.

3

10 lecciones | 10 semanas | 1 proyecto integrador

Repasa, practica y registra cada ejecución en tu Cuadernillo-Diario.

Antes de empezar

Escribir código pythónico y profesional: generadores, decoradores, tipado, concurrencia, logging, consumo de APIs y empaquetado.

Cada lección tiene la misma estructura: explicación breve, un ejemplo para escribir y ejecutar, la salida real que produce, consejos, un ejercicio y una autoevaluación (respuestas al final). Desde este nivel, cada lección tiene además un **ejercicio con lente** de tu ruta de especialización, reunidos en el anexo final.

Además, cada lección tiene tres apartados de profundización (**el concepto a fondo**, **el ejemplo línea por línea** y **errores frecuentes**) y una **práctica evaluable**: un ejercicio que escribes en la app y se corrige automáticamente con pruebas. Cada práctica es una pieza del proyecto del nivel, que se va armando solo a medida que avanzas.

Método: lee en la app, escribe el ejemplo a mano, predice la salida, ejecútalo y anota el resultado en una ficha de **Registro de ejecución** del Cuadernillo-Diario. Cuando apruebes la práctica, regístrala en la **Bitácora de prácticas**: intentos, pistas usadas, el error que más te costó y qué aprendiste.

Tu plan

Semana	Temas	Misión de la semana
1	1. Iteradores y generadores 2. Decoradores	Escribe un decorador <code>@reintentar(veces=3)</code> que vuelva a llamar a la función si lanza una excepción.
2	3. Context managers 4. Type hints y dataclasses	Convierte la clase Tarea de tu gestor en una dataclass con tipos y verifica el proyecto con mypy.
3	5. collections, itertools y functools	Con Counter, encuentra las 5 palabras más frecuentes de un texto largo, ignorando mayúsculas y signos.
4	6. Expresiones regulares	Extrae todos los correos electrónicos y números de teléfono de un texto.
5	7. Concurrencia y asyncio	Simula la descarga de 10 archivos con tiempos aleatorios y compara el tiempo secuencial vs. con gather.
6	8. Logging profesional	Agrega logging a tu gestor de tareas: INFO para acciones, WARNING para entradas inválidas y un archivo de log.
7	9. APIs y HTTP	Consulta una API pública de clima o tipo de cambio y muestra el dato formateado; maneja errores de red con elegancia.
8	10. Empaquetado y distribución	Empaqueta tus utilidades favoritas como paquete instalable con un comando de consola.
9	Proyecto integrador: Monitor asíncrono de precios	Construir, probar y documentar el proyecto
10	Proyecto integrador: Monitor asíncrono de precios	Construir, probar y documentar el proyecto

Ritmo sugerido: 30 a 45 minutos al día, 5 días por semana. Lo importante es no cortar la racha.

1 Iteradores y generadores

Un iterador entrega valores de uno en uno con `next()`. Todo lo que recorres con `for` es iterable.

Una función con `yield` es un generador: produce valores bajo demanda y recuerda dónde quedó. No guarda todo en memoria, así que puede procesar archivos enormes o secuencias infinitas.

Las expresiones generadoras (`x for x in ...`) son la versión compacta.

El concepto a fondo

Un iterable es cualquier cosa que se puede recorrer con `for`; un iterador es el objeto que va entregando los valores de a uno con `next()` y que lanza `StopIteration` cuando se agota. `for` hace todo eso por ti.

Una función con `yield` es un generador: al llamarla no se ejecuta, devuelve un iterador. Cada `next()` corre el código hasta el siguiente `yield`, entrega el valor y se pausa recordando todo su estado. La próxima vez continúa desde ahí.

La gran ventaja es la evaluación perezosa: los valores se producen a medida que se piden. Puedes procesar un archivo de millones de líneas usando la memoria de una sola, o representar secuencias infinitas.

Las expresiones generadoras (`(x * x for x in datos)`) son la versión compacta. Funciones como `sum`, `max` o `any` las aceptan directamente: `sum(p.precio for p in productos)` no crea ninguna lista intermedia.

Ejemplo

```
1 def fibonacci(limite):
2     a, b = 0, 1
3     while a < limite:
4         yield a
5         a, b = b, a + b
6
7 print(list(fibonacci(50)))
8
9 gen = (x * x for x in range(3))
10 print(next(gen), next(gen), next(gen))
```

Salida real

```
[0, 1, 1, 2, 3, 5, 8, 13, 21, 34]
0 1 4
```

El ejemplo, línea por línea

1. `fibonacci` usa `yield`: cada valor se calcula recién cuando se pide.
2. `list(...)` consume el generador hasta que llega al límite.
3. La expresión generadora entrega 0, 1 y 4 con cada `next`.

Consejos

- Un generador se consume una sola vez: si lo necesitas de nuevo, vuelve a crearlo.
- Encadena generadores para crear pipelines de datos que usan muy poca memoria.

Ejercicio

Crea un generador que lea un archivo de log línea por línea y entregue solo las líneas con 'ERROR'.

Errores frecuentes

Error frecuente	Cómo resolverlo
El generador quedó vacío la segunda vez	Un generador se recorre una sola vez. Si necesitas recorrerlo de nuevo, vuelve a llamar a la función.
TypeError: 'generator' object is not subscriptable	Un generador no tiene índices. Conviértelo con <code>list()</code> si necesitas acceder por posición.

Práctica evaluable

Leer precios sin cargar todo en memoria · Pieza del proyecto: **Lectura perezosa con generadores**

Las tiendas envían líneas con el formato `producto;precio` (precio con coma decimal). Escribe dos generadores:

- `leer_precios(lineas)`: entrega tuplas (`producto`, `precio_float`) y saltea líneas vacías, comentarios que empiezan con `#` y líneas con precio inválido.
- `en_lotes(iterable, n)`: entrega listas de hasta `n` elementos; el último lote puede ser más corto.

Ambos deben ser perezosos: no pueden leer todo antes de entregar el primer valor.

Plantilla para empezar

```

1  def leer_precios(lineas):
2      """
3      >>> list(leer_precios(["cable;3,20", "", "# nota", "roto", "pintura;12,5"]))
4      [('cable', 3.2), ('pintura', 12.5)]
5      """
6
7
8  def en_lotes(iterable, n):
9      """
10     >>> list(en_lotes(range(5), 2))
11     [[0, 1], [2, 3], [4]]
12     """

```

Escríbela y evalúala en la app (o en tu IDE). Las pistas y la solución están al final del manual. Al aprobarla, anótala en la **Bitácora de prácticas** del Cuadernillo.

Autoevaluación

¿Cuál es la principal ventaja de un generador?

- Permite índices
- Es más seguro
- Evalúa bajo demanda y ahorra memoria
- Es más corto

2 Decoradores

En Python las funciones son objetos: se pueden pasar como argumento y devolver desde otras funciones.

Un decorador es una función que recibe otra función y devuelve una versión mejorada. Se aplica con `@decorador` encima del `def`.

Se usan para registrar llamadas, medir tiempos, cachear resultados, validar permisos o reintentar operaciones.

El concepto a fondo

En Python las funciones son objetos: se pueden guardar en variables, pasar como argumento y devolver desde otras funciones. Una función definida dentro de otra recuerda las variables del entorno donde nació: eso es una clausura.

Un decorador es una función que recibe una función y devuelve otra que la envuelve, agregando comportamiento antes o después. `@registrar` sobre `def sumar` equivale a escribir `sumar = registrar(sumar)`.

`functools.wraps` copia el nombre, la docstring y otros datos de la función original a la envoltura. Sin él, todas las funciones decoradas se llamarían `envoltura`, y la depuración y la documentación se vuelven confusas.

Un decorador con parámetros, como `@reintentar(veces=3)`, tiene un nivel más: una función que recibe la configuración y devuelve el decorador. Son tres funciones anidadas: configuración → decorador → envoltura.

Ejemplo

```
1 import functools
2
3 def registrar(func):
4     @functools.wraps(func)
5     def envoltura(*args, **kwargs):
6         print(f"-> llamando a {func.__name__}{args}")
7         res = func(*args, **kwargs)
8         print(f"<- devolvió {res}")
9         return res
10    return envoltura
11
12 @registrar
13 def sumar(a, b):
14     return a + b
15
16 sumar(2, 3)
17 print(sumar.__name__)
```

Salida real

```
-> llamando a sumar(2, 3)
<- devolvió 5
sumar
```

El ejemplo, línea por línea

1. `registrar` recibe la función y define `envoltura`, que acepta cualquier argumento.
2. La `envoltura` imprime, llama a la original, imprime el resultado y lo devuelve.

3. @registrar reemplaza sumar por la envoltura.
4. Gracias a wraps, sumar.__name__ sigue siendo 'sumar'.

Consejos

- Usa siempre `functools.wraps` para conservar nombre y docstring de la función original.
- Un decorador con parámetros es una función que devuelve un decorador (tres niveles).

Ejercicio

Escribe un decorador `@reintentar(veces=3)` que vuelva a llamar a la función si lanza una excepción.

Errores frecuentes

Error frecuente	Cómo resolverlo
La función decorada devuelve None	La envoltura llamó a la función pero olvidó <code>return</code> resultado.
<code>TypeError: 'NoneType' object is not callable</code>	El decorador olvidó <code>return</code> envoltura, o usaste <code>@reintentar</code> sin paréntesis siendo un decorador con parámetros.

Práctica evaluable

Reintentar ante fallos de red · Pieza del proyecto: Decoradores de robustez

- `contar_llamadas(func)`: decorador que agrega a la función el atributo `llamadas` y lo incrementa en cada llamada (ya está resuelto en la plantilla como ejemplo).
- `reintentar(veces=3, excepciones=(ConnectionError,))`: decorador con parámetros. Si la función lanza una de esas excepciones, vuelve a intentar hasta `veces` intentos en total. Si se agotan, relanza el último error. Cualquier otra excepción se propaga enseguida, sin reintentar.

Ambos deben conservar el nombre y la docstring de la función original con `functools.wraps`.

Plantilla para empezar

```

1 import functools
2
3
4 def contar_llamadas(func):
5     @functools.wraps(func)
6     def envoltura(*args, **kwargs):
7         envoltura.llamadas += 1
8         return func(*args, **kwargs)
9     envoltura.llamadas = 0
10    return envoltura
11
12
13 def reintentar(veces=3, excepciones=(ConnectionError,)):
14     """Decorador con parámetros: configuración -> decorador -> envoltura."""

```

Escríbela y evalúala en la app (o en tu IDE). Las pistas y la solución están al final del manual. Al aprobarla, anótala en la **Bitácora de prácticas** del Cuadernillo.

Autoevaluación

¿Para qué sirve `functools.wraps`?

- a) Conserva el nombre y docstring originales
- b) Acelera la función
- c) Hace la función asíncrona
- d) Captura errores

3 Context managers

Un context manager prepara algo al entrar a un bloque `with` y lo limpia al salir, pase lo que pase.

Se crean con una clase que implementa `__enter__` y `__exit__`, o más fácil con el decorador `@contextmanager` y un `yield`.

Casos típicos: archivos, conexiones a bases de datos, locks, cambios temporales de configuración, medición de tiempos.

El concepto a fondo

Un context manager define qué pasa al entrar y al salir de un bloque `with`. Al salir, la limpieza ocurre siempre, aunque haya un error o un `return` en el medio. Por eso es la forma correcta de manejar archivos, conexiones y bloqueos.

Como clase, implementa `__enter__` (lo que devuelve se asigna a la variable del `as`) y `__exit__` (tipo, valor, traza), que recibe información del error si lo hubo. Si `__exit__` devuelve `True`, el error se suprime; devolviendo `False` o `None`, se propaga.

Con `contextlib.contextmanager` basta un generador: lo que está antes del `yield` es la entrada, el `yield` es el bloque y lo que va en el `finally` es la salida. Es la forma más corta y la más usada para casos simples.

Un solo `with` puede abrir varios context managers separados por comas, que se cierran en orden inverso.

Ejemplo

```

1  from contextlib import contextmanager
2
3  @contextmanager
4  def seccion(titulo):
5      print(f"[inicio] {titulo}")
6      try:
7          yield
8      finally:
9          print(f"[fin] {titulo}")
10
11  with seccion("Carga de datos"):
12      print("procesando...")

```

Salida real

```

[inicio] Carga de datos
procesando...
[fin] Carga de datos

```

El ejemplo, línea por línea

1. `@contextmanager` convierte el generador en context manager.
2. Se imprime el inicio, `yield` cede el control al bloque `with`.
3. El `finally` garantiza el mensaje de fin aunque el bloque falle.

Consejos

- Envuelve el `yield` en `try/finally` para que la limpieza ocurra aunque haya errores.
- `contextlib.suppress(FileNotFoundError)` ignora un error concreto de forma explícita.

Ejercicio

Crea un context manager `cronometro()` que imprima cuánto tardó el bloque.

Errores frecuentes

Error frecuente	Cómo resolverlo
El recurso no se libera cuando hay un error	La limpieza debe ir en un <code>finally</code> (o en <code>__exit__</code>), no después del <code>yield</code> sin protección.
El error desapareció	Tu <code>__exit__</code> devolvió <code>True</code> . Devuelve <code>False</code> para que el error se propague.

Práctica evaluable

Medir y registrar cada operación · Pieza del proyecto: **Context managers**

- **Cronometro:** context manager como clase. Al entrar guarda el tiempo con `time.perf_counter()` y devuelve el propio objeto; al salir guarda en `self.duracion` los segundos transcurridos. No debe ocultar errores.
- **registro_de(nombre, eventos):** context manager con `@contextmanager`. Agrega a la lista `eventos` el texto "inicio lectura" al entrar y "fin lectura" al salir, siempre. Si dentro del bloque ocurre un error, agrega además "error lectura: <mensaje>" antes del fin, y deja que el error se propague.

Plantilla para empezar

```

1  import time
2  from contextlib import contextmanager
3
4
5  class Cronometro:
6      def __enter__(self):
7          pass
8
9      def __exit__(self, tipo, valor, traza):
10         pass
11
12
13  @contextmanager
14  def registro_de(nombre, eventos):
15      eventos.append(f"inicio {nombre}")
16      try:
17          yield
18      finally:
19          pass

```

Escríbela y evalúala en la app (o en tu IDE). Las pistas y la solución están al final del manual. Al aprobarla, anótala en la **Bitácora de prácticas** del Cuadernillo.

Autoevaluación

¿Qué métodos implementa una clase context manager?

- a) `__enter__` y `__exit__`
- b) `__open__` y `__close__`
- c) `__init__` y `__del__`
- d) `__start__` y `__end__`

4 Type hints y dataclasses

Las anotaciones de tipo (`def f(x: int) -> str`) documentan qué espera y devuelve tu código. Herramientas como mypy y los editores las usan para detectar errores antes de ejecutar.

Python no las aplica en tiempo de ejecución: son información para personas y herramientas.

`@dataclass` genera automáticamente `__init__`, `__repr__` y `__eq__` (y orden con `order=True`) a partir de los atributos anotados.

El concepto a fondo

Las anotaciones de tipo (`def f(precio: float) -> bool`) documentan qué espera y qué devuelve tu código. Python no las verifica al ejecutar, pero herramientas como mypy y tu IDE las usan para detectar errores antes de correr el programa y para autocompletar.

`@dataclass` genera automáticamente `__init__`, `__repr__` y `__eq__` a partir de los campos anotados. Una clase de datos de diez líneas reemplaza fácilmente treinta de código repetitivo.

Opciones útiles: `frozen=True` hace los objetos inmutables (ideal para registros que no deben cambiar), `order=True` permite ordenarlos y `field(default_factory=list)` da a cada objeto su propia lista vacía.

Los campos con valor por defecto deben ir después de los obligatorios. Una dataclass puede tener métodos comunes, como cualquier clase.

Ejemplo

```
1 from dataclasses import dataclass, field
2
3 @dataclass(order=True)
4 class Producto:
5     precio: float
6     nombre: str = field(compare=False)
7     stock: int = 0
8
9 items = [Producto(9.5, "cable"), Producto(3.2, "tornillo", 100)]
10 print(sorted(items)[0])
11 print(items[0] == Producto(9.5, "otro"))
```

Salida real

```
Producto(precio=3.2, nombre='tornillo', stock=100)
True
```

El ejemplo, línea por línea

1. `order=True` genera comparaciones; se ordena por los campos en orden, empezando por precio.

2. `field(compare=False)` excluye el nombre de las comparaciones.
3. Por eso dos productos con igual precio y stock son iguales aunque cambie el nombre.

Consejos

- `@dataclass(frozen=True)` crea objetos inmutables, perfectos como claves de diccionario.
- Usa `list[int]`, `dict[str, float]` y `X | None` (Python 3.10+).

Ejercicio

Convierte la clase Tarea de tu gestor en una dataclass con tipos y verifica el proyecto con mypy.

Errores frecuentes

Error frecuente	Cómo resolverlo
<code>TypeError: non-default argument follows default argument</code>	Pusiste un campo sin valor por defecto después de uno que sí tiene. Reordénalos.
<code>FrozenInstanceError: cannot assign to field</code>	El objeto es <code>frozen=True</code> : crea uno nuevo con <code>dataclasses.replace(obj, campo=valor)</code> .

Práctica evaluable

Modelar productos con dataclasses · Pieza del proyecto: Modelos de datos

- `Producto` (dataclass): `nombre: str`, `precio: float`, `umbral: float = 0.0`, `tienda: str = "web"`, y el método `en_oferta()` -> `bool`, que es `True` si el precio es menor o igual al umbral.
- `Lectura` (dataclass inmutable): `producto: str`, `precio: float`, `fecha: str`.
- `mas_barato(productos)`: el producto de menor precio, o `None` si la lista está vacía.

Anota los tipos en todos los campos y funciones.

Plantilla para empezar

```

1  from dataclasses import dataclass
2
3
4  @dataclass
5  class Producto:
6      nombre: str
7      # completa los campos y el método en_oferta
8
9
10 # Lectura: dataclass inmutable
11
12
13 def mas_barato(productos: list) -> "Producto | None":
14     """El producto de menor precio, o None si no hay productos."""

```

Escríbela y evalúala en la app (o en tu IDE). Las pistas y la solución están al final del manual. Al aprobarla, anótala en la **Bitácora de prácticas** del Cuadernillo.

Autoevaluación

Si el hint dice int y pasas un str, en ejecución Python...

- a) Lo convierte
- b) No compila
- c) No lo impide: los hints no se aplican en runtime
- d) Lanza TypeError

5**collections, itertools y functools**

collections trae estructuras especializadas: Counter (contar), defaultdict (valores por defecto), deque (colas rápidas), namedtuple.

itertools ofrece bloques para iterar: count, cycle, islice, chain, groupby, combinations, product.

functools tiene herramientas funcionales: lru_cache (memoización), partial, reduce.

El concepto a fondo

collections trae estructuras especializadas: Counter cuenta elementos, defaultdict crea automáticamente un valor inicial para claves nuevas (adiós al `if clave not in d`), deque es una cola rápida por ambos extremos (con `maxlen` guarda solo los últimos N) y namedtuple da nombres a las posiciones de una tupla.

itertools combina iteradores sin crear listas: islice toma una porción, count cuenta sin fin, chain encadena, groupby agrupa elementos consecutivos y combinations genera combinaciones.

functools trabaja con funciones: lru_cache memoriza resultados para no recalcularlos, partial fija algunos argumentos de antemano y reduce acumula una secuencia en un solo valor.

Antes de escribir un bucle con contadores y diccionarios a mano, pregúntate si alguno de estos módulos ya lo resuelve: el resultado suele ser más corto, más rápido y más claro.

Ejemplo

```

1  from collections import Counter, defaultdict
2  from itertools import islice, count
3  from functools import lru_cache
4
5  print(Counter("abracadabra").most_common(2))
6
7  grupos = defaultdict(list)
8  for palabra in ["sol", "mar", "sal", "luz"]:
9      grupos[palabra[0]].append(palabra)
10 print(dict(grupos))
11
12 print(list(islice(count(10, 5), 4)))
13
14 @lru_cache(maxsize=None)
15 def fib(n):
16     return n if n < 2 else fib(n - 1) + fib(n - 2)
17 print(fib(80))

```

Salida real

```
[('a', 5), ('b', 2)]
{'s': ['sol', 'sal'], 'm': ['mar'], 'l': ['luz']}
[10, 15, 20, 25]
23416728348467685
```

El ejemplo, línea por línea

1. `Counter(...).most_common(2)` da las dos letras más frecuentes.
2. `defaultdict(list)` crea una lista vacía la primera vez que aparece cada inicial.
3. `islice(count(10, 5), 4)` toma los 4 primeros de 10, 15, 20...
4. `lru_cache` hace instantáneo el Fibonacci recursivo.

Consejos

- Antes de escribir un algoritmo, revisa si `itertools` ya lo tiene.
- `lru_cache` convierte un fibonacci exponencial en lineal con una línea.

Ejercicio

Con `Counter`, encuentra las 5 palabras más frecuentes de un texto largo, ignorando mayúsculas y signos.

Errores frecuentes

Error frecuente	Cómo resolverlo
<code>Counter</code> no cuenta lo que esperabas	<code>Counter("hola")</code> cuenta letras. Para contar palabras pásale una lista: <code>Counter(texto.split())</code> .
<code>groupby</code> agrupa mal	<code>groupby</code> solo agrupa elementos consecutivos: ordena antes por la misma clave.

Práctica evaluable

Analizar precios de varias tiendas · Pieza del proyecto: Análisis con `collections`

Cada lectura es una tupla (producto, tienda, precio). Escribe:

- `mejor_precio(lecturas)`: diccionario {producto: (tienda, precio)} con la oferta más barata de cada producto.
- `victorias_por_tienda(lecturas)`: diccionario {tienda: cantidad} de productos en los que cada tienda tiene el mejor precio (usa `Counter`).
- `tienda_ganadora(lecturas)`: el nombre de la tienda con más victorias (usa `most_common`).
- `ultimos(precios, n)`: lista con los últimos `n` precios ($n \geq 1$). En tu IDE podrías usar `deque(precios, maxlen=n)`; aquí resuélvelo con un rebanado.

Plantilla para empezar

```

1  from collections import Counter, defaultdict
2
3
4  def mejor_precio(lecturas):
5      """{producto: (tienda, precio)} con el precio más bajo de cada producto."""
6
7
8  def victorias_por_tienda(lecturas):
9      """{tienda: cantidad de productos donde es la más barata}."""
10
11
12  def tienda_ganadora(lecturas):
13      """La tienda con más victorias."""
14
15
16  def ultimos(precios, n):
17      """Los últimos n precios (n >= 1)."""

```

Escríbela y evalúala en la app (o en tu IDE). Las pistas y la solución están al final del manual. Al aprobarla, anótala en la **Bitácora de prácticas** del Cuadernillo.

Autoevaluación

Counter("hola")["z"] devuelve...

- a) False
- b) None
- c) KeyError
- d) 0

6 Expresiones regulares

Las regex describen patrones de texto. `re.search` busca la primera coincidencia, `re.findall` todas, `re.sub` reemplaza.

Piezas clave: `\d` dígito, `\w` letra/número, `\s` espacio, `+` uno o más, `*` cero o más, `{n}` exactamente `n`, `[]` conjunto, `()` grupo.

Usa strings crudos `r"...` para que las barras invertidas no se interpreten dos veces.

El concepto a fondo

Una expresión regular describe un patrón de texto. Las piezas básicas: `\d` (dígito), `\w` (letra, número o guion bajo), `\s` (espacio), `.` (cualquier carácter), `[A-Z]` (un carácter del rango), `^` y `$` (inicio y fin) y `\b` (borde de palabra).

Los cuantificadores dicen cuántas veces: `+` (una o más), `*` (cero o más), `?` (opcional) y `{2,4}` (entre 2 y 4). Los paréntesis capturan grupos que luego recuperas; `(?:...)` agrupa sin capturar.

Funciones de `re`: `search` encuentra la primera coincidencia (o `None`), `findall` devuelve todas, `sub` reemplaza y `fullmatch` exige que todo el texto coincida. Escribe los patrones como strings crudos, `r"...`, para que las barras invertidas no se interpreten dos veces.

Las regex son potentes pero difíciles de leer. Comenta las complejas, pruébalas con varios ejemplos (regex101.com ayuda muchísimo) y no las uses donde alcanza con `split`, `startswith` o `in`.

Ejemplo

```
1 import re
2
3 texto = "Pedidos: A-102 el 03/10/2026, B-77 el 15/11/2026"
4 print(re.findall(r"[A-Z]-\d+", texto))
5 print(re.findall(r"(\d{2})/(\d{2})/(\d{4})", texto))
6 print(re.sub(r"\d{4}", "AAAA", texto))
```

Salida real

```
['A-102', 'B-77']
[('03', '10', '2026'), ('15', '11', '2026')]
Pedidos: A-102 el 03/10/AAAA, B-77 el 15/11/AAAA
```

El ejemplo, línea por línea

1. `[A-Z]-\d+` captura una letra, un guion y uno o más dígitos.
2. Con tres grupos, `findall` devuelve tuplas (día, mes, año).
3. `sub` reemplaza cada año de cuatro dígitos por 'AAAA'.

Consejos

- Prueba tus patrones en regex101.com antes de ponerlos en el código.
- Si el patrón se usa muchas veces, compílalo con `re.compile`.

Ejercicio

Extrae todos los correos electrónicos y números de teléfono de un texto.

Errores frecuentes

Error frecuente	Cómo resolverlo
<code>re.error: bad escape</code>	Olvidaste la <code>r</code> del string crudo: escribe <code>r"\d+"</code> .
<code>AttributeError: 'NoneType' object has no attribute 'group'</code>	<code>search</code> no encontró nada y devolvió <code>None</code> . Comprueba el resultado antes de usar <code>.group()</code> .

Práctica evaluable

Extraer precios de textos · Pieza del proyecto: Extracción con regex

Los avisos de las tiendas usan el formato local: punto para los miles y coma para los decimales.

- `extraer_precio(texto)`: el primer precio del texto como float. "\$ 1.234,50" → 1234.5; "99,90 ARS" → 99.9; "Envío 2.500" → 2500.0. Si no hay número, `None`.
- `extraer_codigos(texto)`: lista de códigos de 2 a 4 mayúsculas, guion y 2 a 6 dígitos, como "SKU-1234".
- `normalizar_espacios(texto)`: reemplaza cualquier grupo de espacios, tabulaciones o saltos de línea por un solo espacio y quita los de los bordes.

Plantilla para empezar

```

1  import re
2
3
4  def extraerPrecio(texto):
5      """
6      >>> extraerPrecio("Precio: $ 1.234,50")
7      1234.5
8      """
9
10
11 def extraer_codigos(texto):
12     """
13     >>> extraer_codigos("SKU-1234, ab-12 y TUB-07")
14     ['SKU-1234', 'TUB-07']
15     """
16
17
18 def normalizar_espacios(texto):
19     """
20     >>> normalizar_espacios(" hola \n mundo ")
21     'hola mundo'
22     """

```

Escríbela y evalúala en la app (o en tu IDE). Las pistas y la solución están al final del manual. Al aprobarla, anótala en la **Bitácora de prácticas** del Cuadernillo.

Autoevaluación

¿Qué significa `\d+`?

- a) Un decimal
- b) Cualquier carácter
- c) Uno o más dígitos
- d) Un dígito

7 Concurrency y asyncio

Tres caminos: `threading` para tareas que esperan E/S (red, disco); `multiprocessing` para cálculos pesados de CPU (esquiva el GIL); `asyncio` para miles de operaciones de E/S concurrentes.

En `asyncio`, `async def` define corrutinas y `await` cede el control mientras se espera. `asyncio.gather` ejecuta varias a la vez.

`concurrent.futures` ofrece una interfaz común para hilos y procesos con `ThreadPoolExecutor` y `ProcessPoolExecutor`.

El concepto a fondo

Concurrency es avanzar varias tareas a la vez. Hay tres caminos: `threading` (hilos) para tareas que esperan entrada/salida, `multiprocessing` para cálculos pesados que necesitan varios núcleos, y `asyncio` para miles de esperas de red con un solo hilo.

En `asyncio`, una función `async def` es una corrutina: al llamarla no se ejecuta, se crea una tarea pendiente. `await` pausa la corrutina mientras espera (una respuesta de red, un `sleep`) y le cede el turno al bucle de eventos, que mientras tanto avanza otras tareas.

`asyncio.gather(*tareas)` lanza varias corrutinas juntas y espera a todas: el tiempo total es el de la más lenta, no la suma. `asyncio.Semaphore(n)` limita cuántas corren a la vez, para no saturar a un servidor. `asyncio.run(main())` arranca todo.

En la práctica vas a calcular, sin ejecutar nada en paralelo, cuánto tardaría cada estrategia. Es exactamente el razonamiento que hace falta para decidir cómo consultar muchas tiendas: entender el planificador antes de usarlo. El código asíncrono real lo escribes en tu IDE con el ejercicio de la lección.

Ejemplo

```

1  import asyncio
2
3  async def descargar(nombre, segundos):
4      await asyncio.sleep(segundos)
5      return f"{nombre} listo ({segundos}s)"
6
7  async def main():
8      tareas = [descargar("A", 0.3), descargar("B", 0.1), descargar("C", 0.2)]
9      for r in await asyncio.gather(*tareas):
10         print(r)
11
12  asyncio.run(main())

```

Salida real

```

A listo (0.3s)
B listo (0.1s)
C listo (0.2s)

```

El ejemplo, línea por línea

1. `descargar` es una corrutina: `await asyncio.sleep` simula una espera de red.
2. `gather` lanza las tres a la vez; el total es 0,3 s, no 0,6 s.
3. Los resultados vuelven en el orden en que se pasaron las tareas, no en el que terminaron.

Consejos

- Las tres tareas tardan 0.3 s en total, no 0.6 s: esperan en paralelo.
- Nunca uses `time.sleep` dentro de código `async`: bloquea todo. Usa `await asyncio.sleep`.

Ejercicio

Simula la descarga de 10 archivos con tiempos aleatorios y compara el tiempo secuencial vs. con `gather`.

Errores frecuentes

Error frecuente	Cómo resolverlo
<code>RuntimeWarning: coroutine was never awaited</code>	Llamaste a una función <code>async</code> sin <code>await</code> . Corrutina llamada sin <code>await</code> = tarea que nunca se ejecuta.
Usar <code>time.sleep</code> dentro de <code>async</code>	<code>time.sleep</code> bloquea todo el bucle de eventos. Dentro de corrutinas usa <code>await asyncio.sleep</code> .

Práctica evaluable**Simula el planificador de tareas** · Pieza del proyecto: **Planificación de consultas concurrentes**

Cada consulta a una tienda tarda un tiempo conocido (en segundos). Calcula cuánto tardaría consultarlas todas con cada estrategia:

- `tiempo_secuencial(duraciones)`: una después de otra (como un `for` con `await` dentro).
- `tiempo_concurrente(duraciones)`: todas a la vez (como `asyncio.gather`).
- `tiempo_con_limite(duraciones, limite)`: como `gather` con un `Semaphore(limite)`. Las tareas arrancan en orden y cada una toma el primer lugar que se libera.

Con una lista vacía, las tres devuelven 0.

Plantilla para empezar

```

1  import heapq
2
3
4  def tiempo_secuencial(duraciones):
5      """[3, 1, 2] -> 6"""
6
7
8  def tiempo_concurrente(duraciones):
9      """[3, 1, 2] -> 3"""
10
11
12 def tiempo_con_limite(duraciones, limite):
13     """
14     [3, 1, 2] con limite 2 -> 3
15     t=0: arrancan 3 y 1 · t=1: termina la de 1 y arranca la de 2 · t=3: terminan
16     ↪ ambas.
17     Pista: guarda en un heap el momento en que se libera cada lugar.
18     """

```

Escríbela y evalúala en la app (o en tu IDE). Las pistas y la solución están al final del manual. Al aprobarla, anótala en la **Bitácora de prácticas** del Cuadernillo.

Autoevaluación

Para un cálculo pesado de CPU en CPython conviene...

- `asyncio`
- `multiprocessing`
- `threading`
- un bucle `while`

8 Logging profesional

logging reemplaza a los print de depuración: tiene niveles (DEBUG, INFO, WARNING, ERROR, CRITICAL), formato configurable y destinos múltiples (consola, archivo, servicios).

Cada módulo crea su logger con `logging.getLogger(__name__)`; la configuración se hace una sola vez al inicio de la aplicación.

Pasa los datos como argumentos (`log.info("%d items", n)`) en lugar de f-strings: solo se formatean si el mensaje realmente se muestra.

El concepto a fondo

logging reemplaza a los print de depuración. Cada mensaje tiene un nivel (DEBUG, INFO, WARNING, ERROR, CRITICAL) y puedes elegir desde qué nivel mostrar sin tocar el código: en desarrollo ves todo, en producción solo lo importante.

Un logger se obtiene con `logging.getLogger("nombre")`, normalmente uno por módulo. Los handlers deciden adónde van los mensajes (consola, archivo, archivo rotativo) y los formatters cómo se ven (fecha, nivel, módulo, mensaje).

Pasa los datos como argumentos y no con f-strings: `log.info("Stock: %d", n)`. Así el texto solo se arma si el mensaje realmente se va a mostrar.

Diseñar qué es WARNING y qué es ERROR es una decisión de negocio. En el monitor, una variación grande de precio merece atención (WARNING); un cambio chico es informativo (INFO).

Ejemplo

```
1 import logging, sys
2
3 logging.basicConfig(level=logging.INFO, stream=sys.stdout,
4                     format="%(levelname)s | %(name)s | %(message)s")
5 log = logging.getLogger("inventario")
6
7 log.debug("esto no se ve")
8 log.info("Stock cargado: %d productos", 120)
9 log.warning("Producto %s bajo mínimo", "TUBO-12")
```

Salida real

```
INFO | inventario | Stock cargado: 120 productos
WARNING | inventario | Producto TUBO-12 bajo mínimo
```

El ejemplo, línea por línea

1. `basicConfig` fija el nivel mínimo (INFO), la salida y el formato.
2. El DEBUG no se muestra porque está por debajo de INFO.
3. Los `%d` y `%s` se completan con los argumentos.

Consejos

- `log.exception("...")` dentro de un except registra el traceback completo.
- Usa `RotatingFileHandler` para que los archivos de log no crezcan sin límite.

Ejercicio

Agrega logging a tu gestor de tareas: INFO para acciones, WARNING para entradas inválidas y un archivo de log.

Errores frecuentes

Error frecuente	Cómo resolverlo
No aparece ningún mensaje	El nivel configurado es más alto que el de tus mensajes, o basicConfig se llamó después de crear handlers.
Mensajes duplicados	Agregaste el mismo handler más de una vez, o el logger propaga al raíz que también tiene handler.

Práctica evaluable**Registrar cambios de precio · Pieza del proyecto: Registro de cambios con niveles**

- `variacion_pct(antes, ahora)`: variación porcentual redondeada a 1 decimal. Si antes es 0, lanza `ValueError`.
- `nivel_para(cambio_pct)`: "DEBUG" si no cambió, "WARNING" si la variación es de 20 % o más (subida o bajada) e "INFO" en otro caso.
- `registrar_cambio(log, producto, antes, ahora)`: llama al método del logger según el nivel (`log.info`, `log.warning`...) con el mensaje "cable: 10.00 -> 8.00 (-20.0%)" y devuelve el nivel usado.

El evaluador te pasa un logger falso que anota lo que recibe: así se prueban los logs sin configurar nada.

Plantilla para empezar

```

1 def variacion_pct(antes, ahora):
2     """10 -> 8 da -20.0"""
3
4
5 def nivel_para(cambio_pct):
6     """0 -> DEBUG; |cambio| >= 20 -> WARNING; si no, INFO."""
7
8
9 def registrar_cambio(log, producto, antes, ahora):
10     """Registra con el nivel adecuado y devuelve el nivel."""
11     # Pista: getattr(log, "warning") es lo mismo que log.warning

```

Escríbela y evalúala en la app (o en tu IDE). Las pistas y la solución están al final del manual. Al aprobarla, anótala en la **Bitácora de prácticas** del Cuadernillo.

Autoevaluación

Con `level=INFO`, ¿se muestran los mensajes DEBUG?

- Solo en archivos
- Depende del SO
- No
- Sí

9 APIs y HTTP

Una API web permite que programas conversen por HTTP. Los verbos principales: GET (leer), POST (crear), PUT/PATCH (actualizar), DELETE (borrar).

La librería `requests` (o `httpx`, que además soporta `async`) simplifica las llamadas. La mayoría de APIs responde en JSON.

Buenas prácticas: siempre `timeout`, verificar el código de estado con `raise_for_status()`, guardar claves en variables de entorno y respetar los límites de uso.

El concepto a fondo

Una API web permite que dos programas conversen por HTTP. El cliente envía una petición con un verbo (GET leer, POST crear, PUT o PATCH actualizar, DELETE borrar), una URL y, a veces, un cuerpo JSON; el servidor responde con un código de estado y datos.

Los códigos de estado se leen por centenas: 2xx éxito, 3xx redirección, 4xx error del cliente (404 no encontrado, 401 sin autorización, 429 demasiadas peticiones) y 5xx error del servidor. Los 429 y 5xx suelen ser temporales y merecen un reintento; los 4xx no.

Los parámetros de consulta van en la URL después de `?`: `?producto=cable+2mm&pagina=2`. `urllib.parse.urlencode` los arma escapando espacios y símbolos correctamente.

Con `requests`: `requests.get(url, params=..., timeout=10)`, `resp.raise_for_status()` lanza error ante 4xx y 5xx, y `resp.json()` convierte el cuerpo. Pon siempre un `timeout`: sin él, un servidor colgado congela tu programa para siempre.

Ejemplo

```
1 import requests
2
3 resp = requests.get("https://api.github.com/repos/python/cpython", timeout=10)
4 resp.raise_for_status()
5 repo = resp.json()
6 print(repo["full_name"], "-", repo["stargazers_count"], "estrellas")
```

Resultado esperado

```
python/cpython - 6xxxx estrellas (el número cambia con el tiempo)
```

El ejemplo, línea por línea

1. `requests.get` hace la petición con un límite de 10 segundos.
2. `raise_for_status` convierte un error HTTP en excepción.
3. `resp.json()` devuelve el diccionario con los datos del repositorio.

Consejos

- Nunca escribas una API key directamente en el código: usa `os.environ` o un archivo `.env`.
- Usa `requests.Session()` cuando hagas muchas llamadas al mismo servidor.

Ejercicio

Consulta una API pública de clima o tipo de cambio y muestra el dato formateado; maneja errores de red con elegancia.

Errores frecuentes

Error frecuente	Cómo resolverlo
<code>requests.exceptions.Timeout</code>	El servidor no respondió a tiempo. Es un error temporal: reintenta con espera creciente.
<code>JSONDecodeError</code> al llamar a <code>.json()</code>	La respuesta no era JSON (por ejemplo, una página de error HTML). Revisa el código de estado antes.

Práctica evaluable**Hablar con la API de una tienda · Pieza del proyecto: Cliente de API**

- `construir_url(base, **params)`: arma la URL con los parámetros de consulta, ignorando los que valgan `None`. Sin parámetros, devuelve la base tal cual.

```
construir_url("https://api.ej.com/precios", producto="cable 2mm", pagina=2, tienda=None)
→ "https://api.ej.com/precios?producto=cable+2mm&pagina=2"
```

- `procesar_respuesta(estado, cuerpo)`: con 200 devuelve la lista "productos" del JSON; con 404 devuelve `[]`; con 429 o cualquier 5xx lanza `ConnectionError` (temporal: tu decorador reintentar lo aprovechará); con cualquier otro código lanza `ValueError`.

Plantilla para empezar

```

1 import json
2 from urllib.parse import urlencode
3
4
5 def construir_url(base, **params):
6     """URL con parámetros de consulta, sin los None."""
7
8
9 def procesar_respuesta(estado, cuerpo):
10     """Interpreta el código de estado y el cuerpo JSON."""
```

Escríbela y evalúala en la app (o en tu IDE). Las pistas y la solución están al final del manual. Al aprobarla, anótala en la **Bitácora de prácticas** del Cuadernillo.

Autoevaluación

¿Por qué pasar `timeout` a `requests.get`?

- Para ir más rápido
- Para que el programa no quede colgado indefinidamente
- Para cachear
- Es obligatorio

10 Empaquetado y distribución

`pyproject.toml` es el estándar moderno para describir un proyecto: nombre, versión, dependencias y herramienta de construcción.

La estructura recomendada es `src/mi_paquete/` más una carpeta `tests/`. Así evitas importar por accidente código no instalado.

Con `python -m build` generas el paquete y con `twine upload` lo publicas en PyPI. Puedes definir comandos de consola en `[project.scripts]`.

El concepto a fondo

`pyproject.toml` es el estándar para describir un proyecto de Python: nombre, versión, versión mínima de Python, dependencias, scripts de consola y herramientas (como `ruff` o `pytest`). Herramientas como `pip`, `uv` o `hatch` lo leen para instalar y construir el paquete.

Se recomienda la estructura `src/`: el código vive en `src/mi_paquete/`, lo que obliga a instalarlo (`pip install -e .`) para probarlo y evita importar por accidente desde la carpeta equivocada.

El versionado semántico usa MAYOR.MENOR.PARCHE: el parche corrige errores sin cambiar nada más, el menor agrega funciones compatibles y el mayor rompe compatibilidad. Al subir una parte, las de la derecha vuelven a 0: de 1.4.2 a menor da 1.5.0.

Comparar versiones como texto es un error clásico: `"1.10.0" < "1.9.0"` es `True` porque compara carácter por carácter. Hay que convertir cada parte a número y comparar tuplas.

Ejemplo (toml)

```
1 [project]
2 name = "pythonia-utils"
3 version = "0.1.0"
4 requires-python = ">=3.10"
5 dependencies = ["requests>=2.31"]
6
7 [project.scripts]
8 pythonia = "pythonia_utils.cli:main"
9
10 [build-system]
11 requires = ["hatchling"]
12 build-backend = "hatchling.build"
```

Resultado esperado

```
$ pip install -e .
$ pythonia --help
```

El ejemplo, línea por línea

1. `[project]` define nombre, versión, Python mínimo y dependencias.
2. `[project.scripts]` crea el comando `pythonia` que ejecuta la función `main`.
3. `[build-system]` indica con qué herramienta se construye el paquete.

Consejos

- Versiona con SemVer: MAYOR.MENOR.PARCHES.
- `pip install -e .` instala en modo editable: tus cambios se reflejan sin reinstalar.

Ejercicio

Empaqueta tus utilidades favoritas como paquete instalable con un comando de consola.

Errores frecuentes

Error frecuente	Cómo resolverlo
ModuleNotFoundError al ejecutar tests	Con estructura src/, instala el paquete en modo editable: <code>pip install -e ..</code>
Comparar versiones como texto	"1.10" es menor que "1.9" como texto. Convierte a tuplas de enteros.

Práctica evaluable

Versionado semántico · Pieza del proyecto: **Versionado del monitor**

- `comparar_versiones(a, b)`: devuelve -1 si a es anterior, 0 si son iguales y 1 si a es posterior. Compara número por número: "1.10.0" es posterior a "1.9.3".
- `siguiente_version(version, parte)`: con parte "major", "minor" o "patch" devuelve la versión siguiente, poniendo en 0 las partes de la derecha. Con otra parte, lanza `ValueError`.

Plantilla para empezar

```

1 def comparar_versiones(a, b):
2     """
3     >>> comparar_versiones("1.10.0", "1.9.3")
4     1
5     """
6
7
8 def siguiente_version(version, parte):
9     """
10    >>> siguiente_version("1.4.2", "minor")
11    '1.5.0'
12    """

```

Escríbela y evalúala en la app (o en tu IDE). Las pistas y la solución están al final del manual. Al aprobarla, anótala en la **Bitácora de prácticas** del Cuadernillo.

Autoevaluación

¿Qué archivo es el estándar moderno para configurar un paquete?

- requirements.txt
- setup.cfg
- pyproject.toml
- package.json

Proyecto integrador

Monitor asíncrono de precios

Una herramienta que consulta varias APIs en paralelo, guarda el histórico y avisa cuando un valor cruza un umbral.

Paso a paso

1. Modela los datos con dataclasses tipadas.
2. Consulta las fuentes en paralelo con asyncio + httpx, con timeout y reintentos (decorador propio).
3. Guarda el histórico en SQLite o CSV usando un context manager.
4. Registra todo con logging (consola + archivo rotativo).
5. Empaquétalo con pyproject.toml y un comando de consola.
6. Extra: notificación por Telegram o correo cuando se cruce el umbral.

Criterios de éxito

Funciona sin errores con entradas normales y extrañas, está dividido en funciones o clases con nombres claros, tiene docstrings y tests donde corresponde, y registraste el proceso en la Plantilla de proyecto del Cuadernillo.

Checklist de cierre

- ☐ Puedo explicar y usar: iteradores y generadores
- ☐ Puedo explicar y usar: decoradores
- ☐ Puedo explicar y usar: context managers
- ☐ Puedo explicar y usar: type hints y dataclasses
- ☐ Puedo explicar y usar: collections, itertools y functools
- ☐ Puedo explicar y usar: expresiones regulares
- ☐ Puedo explicar y usar: concurrencia y asyncio
- ☐ Puedo explicar y usar: logging profesional
- ☐ Puedo explicar y usar: apis y http
- ☐ Puedo explicar y usar: empaquetado y distribución
- ☐ Aprobé las 10 prácticas evaluables y mi proyecto ejecuta la demo
- ☐ Terminé el proyecto "Monitor asíncrono de precios"

Tu proyecto del nivel, pieza a pieza

Monitor de precios

Un monitor que lee precios de varias tiendas, los valida, detecta ofertas, reintenta ante fallos de red y registra los cambios importantes. Cada lección del Nivel 3 aporta una pieza: generadores, decoradores, context managers, dataclasses, collections, regex, concurrencia, logging, APIs y versionado.

Cada práctica evaluable del nivel es una pieza de este programa. En la app, la sección **Mi proyecto** (desde el Árbol o el Inicio) une tus soluciones aprobadas en un solo archivo, `monitor_precios.py`, te deja ejecutar la demo en el navegador cuando están todas y descargarlo en .zip (con un README) para abrirlo en tu IDE. Las piezas pendientes se reemplazan por versiones provisionales, así que puedes descargarlo en cualquier momento.

Las piezas

Lección	Pieza	Práctica
3.1 Iteradores y generadores	Lectura perezosa con generadores	Leer precios sin cargar todo en memoria
3.2 Decoradores	Decoradores de robustez	Reintentar ante fallos de red
3.3 Context managers	Context managers	Medir y registrar cada operación
3.4 Type hints y dataclasses	Modelos de datos	Modelar productos con dataclasses
3.5 collections, itertools y functools	Análisis con collections	Analizar precios de varias tiendas
3.6 Expresiones regulares	Extracción con regex	Extraer precios de textos
3.7 Concurrencia y asyncio	Planificación de consultas concurrentes	Simula el planificador de tareas
3.8 Logging profesional	Registro de cambios con niveles	Registrar cambios de precio
3.9 APIs y HTTP	Cliente de API	Hablar con la API de una tienda
3.10 Empaquetado y distribución	Versionado del monitor	Versionado semántico

El programa principal

Este es el código que Pythonia agrega al final del archivo: no tienes que escribirlo, pero vale la pena leerlo, porque muestra cómo se conectan tus piezas. `demo()` hace un recorrido automático y el bloque `if __name__ == "__main__":` es lo que corre al ejecutar el archivo en tu IDE.

```

1  # ===== Programa principal (escrito por Pythonia, usa tus piezas)
   ↳ =====
2  import random
3
4  DATOS = """# producto;precio  (respuesta simulada de una tienda)
5  cable;3,20
6  pintura;12,50
7  rodillo;6,00
8  dato roto
9  tornillo;0,15
10 """
11
12 LECTURAS = [("cable", "Centro", 3.2), ("cable", "Norte", 3.0), ("pintura",
↳ "Centro", 12.5),
13             ("pintura", "Norte", 13.0), ("rodillo", "Sur", 5.9), ("rodillo",
↳ "Centro", 6.0),
14             ("tornillo", "Norte", 0.15), ("tornillo", "Centro", 0.2)]
15
16
17 class LogPantalla:
18     """Logger mínimo que imprime, para que la demo funcione en cualquier entorno."""
19     def debug(self, m): print("DEBUG  ", m)
20     def info(self, m): print("INFO   ", m)
21     def warning(self, m): print("WARNING", m)
22     def error(self, m): print("ERROR  ", m)
23
24
25 _fallos = {"n": 0}
26
27
28 @reintentar(vec=3)
29 def consultar_tienda():
30     """Simula una tienda que falla las dos primeras veces."""
31     _fallos["n"] += 1
32     if _fallos["n"] < 3:
33         raise ConnectionError("la tienda no responde")
34     return procesar_respuesta(200, '{"productos": [{"cable", 3.0}, {"pintura",
↳ 9.9}, {"rodillo", 6.0}]}')
35
36

```

```

37 def demo():
38     """Recorrido automático por todas tus piezas."""
39     eventos = []
40     with registro_de("lectura", eventos), Cronometro() as crono:
41         lecturas = list(LeerPrecios(DATOS.splitlines()))
42         print("Lecturas válidas:", lecturas)
43         print("Eventos:", eventos, f"| duró {crono.duracion:.4f}s")
44         productos = [Producto(n, p, umbral=round(p * 0.9, 2)) for n, p in lecturas]
45         print("Primer producto:", productos[0])
46         nuevos = dict(consultar_tienda())
47         print(f"La tienda respondió al intento {_fallos['n']}")
48         log = LogPantalla()
49         for p in productos:
50             if p.nombre in nuevos:
51                 registrar_cambio(log, p.nombre, p.precio, nuevos[p.nombre])
52         print("Lotes de 2:", list(en_lotes([p.nombre for p in productos], 2)))
53         print("Mejor precio por producto:", mejor_precio(LECTURAS))
54         print("Tienda ganadora:", tienda_ganadora(LECTURAS),
55               ↪ victorias_por_tienda(LECTURAS))
56         print("Precio en un aviso:", extraer_precio("Oferta imperdible: $ 1.234,50
57               ↪ final"))
58         print("URL de consulta:", construir_url("https://api.tienda.com/precios",
59               ↪ producto="cable 2mm", pagina=1))
60         dur = [0.3, 0.1, 0.2, 0.4]
61         print(f"Consultar 4 tiendas: en secuencia {tiempo_secuencial(dur):.1f}s, todas
62               ↪ juntas "
63               f"{tiempo_concurrente(dur):.1f}s, de a 2 {tiempo_con_limite(dur,
64               ↪ 2):.1f}s")
65         print("Próxima versión del monitor:", siguiente_version("1.4.2", "minor"))
66
67     async def consultar(producto, segundos):
68         import asyncio
69         await asyncio.sleep(segundos)
70         return producto, round(random.uniform(1, 20), 2)
71
72     async def monitorear():
73         """Versión asíncrona real: ejecútala en tu IDE."""
74         import asyncio, logging
75         logging.basicConfig(level=logging.DEBUG, format="%(levelname)-7s %(message)s")
76         log = logging.getLogger("monitor")
77         tareas = [consultar(n, s) for n, s in [("cable", 0.3), ("pintura", 0.1),
78               ↪ ("rodillo", 0.2)]]
79         for producto, precio in await asyncio.gather(*tareas):
80             registrar_cambio(log, producto, 10.0, precio)
81
82 if __name__ == "__main__":
83     import asyncio
84     demo()
85     print("\n— Monitor asíncrono real —")
86     asyncio.run(monitorear())

```

Ejercicios con lente de especialización

Cada concepto del nivel aplicado a tu ruta. Elige la columna de tu ruta principal; si sigues el Modo Pythonista Total, resuelve las tres.

Automatización de oficina

Lección	Ejercicio con lente
1. Iteradores y generadores	Crea un generador que recorra un Excel enorme con <code>iter_rows</code> y entregue solo las filas con stock bajo.
2. Decoradores	Escribe <code>@medir_tiempo</code> y aplícalo a cada paso de tu robot para descubrir cuál tarda más.
3. Context managers	Crea un context manager <code>carpeta_temporal()</code> que cree una carpeta de trabajo y la borre al terminar.
4. Type hints y dataclasses	Modela cada fila de ventas como <code>dataclass</code> tipada y verifica el robot con <code>mypy</code> .
5. collections, itertools y functools	Usa <code>defaultdict</code> y <code>Counter</code> para totalizar ventas por sucursal y hallar los 5 productos más vendidos.
6. Expresiones regulares	Extrae números de factura, identificadores fiscales y montos del texto de correos con expresiones regulares.
7. Concurrencia y asyncio	Descarga en paralelo los reportes de varias sucursales simuladas con <code>asyncio.gather</code> .
8. Logging profesional	Configura un log rotativo con un resumen final: archivos procesados, saltados y con error.
9. APIs y HTTP	Consulta una API de tipo de cambio y agrega una columna convertida a tu planilla consolidada.
10. Empaquetado y distribución	Empaqueta tu robot como comando de consola: <code>robot-reportes --carpeta ./entrada</code> .

Videojuegos

Lección	Ejercicio con lente
1. Iteradores y generadores	Crea un generador infinito de oleadas que aumente la dificultad en cada ronda.
2. Decoradores	Escribe <code>@cooldown(0.5)</code> para que un disparo solo pueda repetirse cada medio segundo.
3. Context managers	Crea un context manager <code>pausa_juego()</code> que detenga música y reloj y los reanude al salir.
4. Type hints y dataclasses	Define la configuración de cada enemigo como <code>dataclass</code> inmutable (<code>frozen=True</code>).
5. collections, itertools y functools	Usa <code>deque</code> para guardar las últimas 10 posiciones del jugador y dibujar una estela.
6. Expresiones regulares	Valida nombres para el ranking con una regex: de 3 a 12 letras o números.
7. Concurrencia y asyncio	Carga los recursos con <code>asyncio</code> mientras muestras una pantalla de carga animada.

Lección	Ejercicio con lente
8. Logging profesional	Registra en un log los eventos de cada partida (nivel, muertes, tiempo) para ajustar la dificultad.
9. APIs y HTTP	Envía el puntaje final a una API de ranking simulada, con timeout y manejo de errores.
10. Empaquetado y distribución	Empaqueta tu juego como paquete instalable con el comando <code>mi-juego</code> .

Bases de datos

Lección	Ejercicio con lente
1. Iteradores y generadores	Lee una tabla de un millón de filas por lotes con un generador y <code>fetchmany</code> .
2. Decoradores	Escribe <code>@transaccional</code> que haga commit o rollback automático alrededor de una función.
3. Context managers	Crea un context manager <code>conexion()</code> que abra, entregue y cierre siempre la conexión.
4. Type hints y dataclasses	Convierte cada fila en una dataclass tipada usando <code>row_factory</code> .
5. collections, itertools y functools	Agrupar resultados por categoría con <code>itertools.groupby</code> y <code>defaultdict</code> .
6. Expresiones regulares	Valida correos y teléfonos con regex antes de insertarlos.
7. Concurrencia y asyncio	Consulta varias bases en paralelo con <code>aiosqlite</code> y <code>asyncio.gather</code> .
8. Logging profesional	Registra con logging las consultas que tarden más de 100 ms.
9. APIs y HTTP	Sincroniza una tabla local con datos de una API pública sin duplicar registros.
10. Empaquetado y distribución	Empaqueta tus utilidades de base de datos como una librería interna.

Solucionario de autoevaluación

- 1. Iteradores y generadores:** c) Evalúa bajo demanda y ahorra memoria. Evaluación perezosa: solo calcula el siguiente valor cuando se pide.
- 2. Decoradores:** a) Conserva el nombre y docstring originales. Sin wraps, sumar.__name__ sería 'envoltura'.
- 3. Context managers:** a) __enter__ y __exit__. with llama a __enter__ al entrar y a __exit__ al salir.
- 4. Type hints y dataclasses:** c) No lo impide: los hints no se aplican en runtime. Los verificadores estáticos (mypy, pyright) son los que lo detectan.
- 5. collections, itertools y functools:** d) 0. Counter devuelve 0 para claves inexistentes en lugar de fallar.
- 6. Expresiones regulares:** c) Uno o más dígitos. \d es un dígito y + significa 'uno o más'.
- 7. Concurrencia y asyncio:** b) multiprocessing. El GIL impide que los hilos ejecuten bytecode en paralelo; los procesos no comparten GIL.
- 8. Logging profesional:** c) No. Se muestran los mensajes del nivel configurado hacia arriba; DEBUG está por debajo de INFO.
- 9. APIs y HTTP:** b) Para que el programa no quede colgado indefinidamente. Sin timeout, una conexión que no responde puede bloquear tu programa para siempre.
- 10. Empaquetado y distribución:** c) pyproject.toml. pyproject.toml (PEP 517/518/621) centraliza la configuración.

Pistas y soluciones de las prácticas

Usa las pistas de a una, y solo después de intentarlo. Hay más de una solución correcta: si la tuya pasa las pruebas, compárala con esta para aprender otra forma de pensarlo.

1. Iteradores y generadores · Leer precios sin cargar todo en memoria

Pista 1: `linea.partition(";")` separa en (antes, separador, después) sin fallar si no hay `;`.

Pista 2: En `en_lotes`, después del `yield` crea una lista nueva vacía; al terminar el `for`, entrega lo que quedó.

```

1  def leer_precios(lineas):
2      """
3      >>> list(leer_precios(["cable;3,20", "", "# nota", "roto", "pintura;12,5"]))
4      [('cable', 3.2), ('pintura', 12.5)]
5      """
6      for linea in lineas:
7          linea = linea.strip()
8          if not linea or linea.startswith("#"):
9              continue
10         producto, _, precio = linea.partition(";")
11         try:
12             yield producto.strip(), float(precio.strip().replace(",", "."))
13         except ValueError:
14             continue
15
16
17  def en_lotes(iterable, n):
18      """
19      >>> list(en_lotes(range(5), 2))
20      [[0, 1], [2, 3], [4]]
21      """
22      lote = []
23      for elemento in iterable:
24          lote.append(elemento)
25          if len(lote) == n:
26              yield lote
27              lote = []
28      if lote:
29          yield lote

```

Por qué funciona: Los dos generadores procesan un elemento a la vez y nunca guardan más de un lote en memoria. Funcionarían igual con un archivo de varios gigas. Por eso el evaluador verifica que solo se consumió lo necesario para entregar el primer valor.

2. Decoradores · Reintentar ante fallos de red

Pista 1: `except excepciones:` acepta directamente una tupla de clases.

Pista 2: Un `raise` solo, dentro del `except`, relanza el error actual con su traceback.

```

1  import functools
2
3
4  def contar_llamadas(func):
5      @functools.wraps(func)
6      def envoltura(*args, **kwargs):
7          envoltura.llamadas += 1
8          return func(*args, **kwargs)
9      envoltura.llamadas = 0
10     return envoltura
11
12
13 def reintentar(veces=3, excepciones=(ConnectionError,)):
14     """Decorador con parámetros: configuración -> decorador -> envoltura."""
15     def decorador(func):
16         @functools.wraps(func)
17         def envoltura(*args, **kwargs):
18             for intento in range(1, veces + 1):
19                 try:
20                     return func(*args, **kwargs)
21                 except excepciones:
22                     if intento == veces:
23                         raise
24             return envoltura
25     return decorador

```

Por qué funciona: Los tres niveles se ven claros: `reintentar` guarda la configuración, `decorador` recibe la función y `envoltura` hace el trabajo. El `return` dentro del `try` corta el bucle apenas hay éxito; el `raise` final solo ocurre en el último intento.

3. Context managers · Medir y registrar cada operación

Pista 1: En `__exit__`, `return False` deja pasar el error.

Pista 2: En el generador: `try: yield` · `except Exception as e:` anota y `raise` · `finally:` anota el fin.

```

1  import time
2  from contextlib import contextmanager
3
4
5  class Cronometro:
6      def __enter__(self):
7          self.inicio = time.perf_counter()
8          return self
9
10     def __exit__(self, tipo, valor, traza):
11         self.duracion = float(time.perf_counter() - self.inicio)
12         return False
13
14
15  @contextmanager
16  def registro_de(nombre, eventos):
17      eventos.append(f"inicio {nombre}")
18      try:
19          yield
20      except Exception as e:
21          eventos.append(f"error {nombre}: {e}")
22          raise
23      finally:
24          eventos.append(f"fin {nombre}")

```

Por qué funciona: Las dos formas hacen lo mismo con distinta sintaxis. El orden `except` → `raise` → `finally` garantiza que el error quede anotado, se propague y que el cierre ocurra igual. Es exactamente lo que hace `open()` por dentro con los archivos.

4. Type hints y dataclasses · Modelar productos con dataclasses

Pista 1: Los campos con valor por defecto van al final.

Pista 2: `min(iterable, key=...)` devuelve el elemento, no el valor.

```

1  from dataclasses import dataclass
2
3
4  @dataclass
5  class Producto:
6      nombre: str
7      precio: float
8      umbral: float = 0.0
9      tienda: str = "web"
10
11     def en_oferta(self) -> bool:
12         return self.precio <= self.umbral
13
14
15  @dataclass(frozen=True)
16  class Lectura:
17      producto: str
18      precio: float
19      fecha: str
20
21
22  def mas_barato(productos: list) -> "Producto | None":
23      """El producto de menor precio, o None si no hay productos."""
24      if not productos:
25          return None
26      return min(productos, key=lambda p: p.precio)

```

Por qué funciona: Con pocas líneas obtienes constructor, representación legible e igualdad. `frozen=True` protege las lecturas históricas de cambios accidentales: un precio registrado no debería poder alterarse.

5. collections, itertools y functools · Analizar precios de varias tiendas

Pista 1: `defaultdict(list)` agrupa las ofertas por producto sin preguntar si la clave existe.

Pista 2: Los últimos `n` con un rebanado: `precios[-n:]`.

```

1  from collections import Counter, defaultdict
2
3
4  def mejor_precio(lecturas):
5      """{producto: (tienda, precio)} con el precio más bajo de cada producto."""
6      por_producto = defaultdict(list)
7      for producto, tienda, precio in lecturas:
8          por_producto[producto].append((tienda, precio))
9      return {prod: min(ofertas, key=lambda o: o[1]) for prod, ofertas in
10             ↪ por_producto.items()}
11
12 def victorias_por_tienda(lecturas):
13     """{tienda: cantidad de productos donde es la más barata}."""
14     return dict(Counter(tienda for tienda, _ in mejor_precio(lecturas).values()))
15
16
17 def tienda_ganadora(lecturas):
18     """La tienda con más victorias."""
19     return Counter(victorias_por_tienda(lecturas)).most_common(1)[0][0]
20
21
22 def ultimos(precios, n):
23     """Los últimos n precios (n >= 1)."""
24     return list(precios[-n:])

```

Por qué funciona: Agrupar con defaultdict, elegir con min(key=...) y contar con Counter resuelve en pocas líneas un análisis que a mano llevaría varios diccionarios auxiliares. Las funciones se apoyan unas en otras, sin repetir lógica.

6. Expresiones regulares · Extraer precios de textos

Pista 1: Patrón de precio: grupos de miles `\d{1,3}(\?:\.\d{3})+` o un número simple `\d+`, con decimales opcionales `(\?:\.\d+)?`.

Pista 2: `\s+` atrapa espacios, tabulaciones y saltos de línea juntos.

```

1 import re
2
3 PRECIO = re.compile(r"\d{1,3}(?:\.\d{3})+(?:,\d+)?|\d+(?:,\d+)?")
4
5
6 def extraer_precio(texto):
7     """
8     >>> extraer_precio("Precio: $ 1.234,50")
9     1234.5
10    """
11    m = PRECIO.search(texto)
12    if m is None:
13        return None
14    return float(m.group().replace(".", "").replace(",", "."))
15
16
17 def extraer_codigos(texto):
18     """
19     >>> extraer_codigos("SKU-1234, ab-12 y TUB-07")
20     ['SKU-1234', 'TUB-07']
21    """
22    return re.findall(r"\b[A-Z]{2,4}-\d{2,6}\b", texto)
23
24
25 def normalizar_espacios(texto):
26     """
27     >>> normalizar_espacios(" hola \n mundo ")
28     'hola mundo'
29    """
30    return re.sub(r"\s+", " ", texto).strip()

```

Por qué funciona: La alternativa | prueba primero el formato con miles y, si no aplica, un número simple. Compilar el patrón con `re.compile` y darle un nombre lo hace reutilizable y más legible. La conversión final adapta el formato local al que entiende `float`.

7. Concurrencia y asyncio · Simula el planificador de tareas

Pista 1: Un heap (heapq) siempre entrega primero el menor: el lugar que se libera antes.

Pista 2: Mientras haya lugares libres, cada tarea termina en `d`; después, en `momento_libre + d`.

```

1  import heapq
2
3
4  def tiempo_secuencial(duraciones):
5      """[3, 1, 2] -> 6"""
6      return sum(duraciones)
7
8
9  def tiempo_concurrente(duraciones):
10     """[3, 1, 2] -> 3"""
11     return max(duraciones, default=0)
12
13
14  def tiempo_con_limite(duraciones, limite):
15     """Simula gather + Semaphore(limite)."""
16     fines = [] # heap: momento en que se libera cada lugar ocupado
17     for d in duraciones:
18         if len(fines) < limite:
19             heapq.heappush(fines, d)
20         else:
21             libre = heapq.heappop(fines)
22             heapq.heappush(fines, libre + d)
23     return max(fines, default=0)

```

Por qué funciona: Es el corazón de un semáforo: hay límite lugares, y cada tarea nueva espera al que se libera primero. El heap guarda los momentos de fin, y el tiempo total es el mayor de ellos. Con esto puedes justificar con números por qué conviene gather con un límite razonable.

8. Logging profesional · Registrar cambios de precio

Pista 1: El formato `+.1f` muestra el signo y un decimal: `+5.0`, `-20.0`.

Pista 2: `getattr(log, nivel.lower())` obtiene el método por su nombre.

```

1  def variacion_pct(antes, ahora):
2      """10 -> 8 da -20.0"""
3      if antes == 0:
4          raise ValueError("No se puede calcular la variación desde un precio 0")
5      return round((ahora - antes) / antes * 100, 1)
6
7
8  def nivel_para(cambio_pct):
9      """0 -> DEBUG; |cambio| >= 20 -> WARNING; si no, INFO."""
10     if cambio_pct == 0:
11         return "DEBUG"
12     if abs(cambio_pct) >= 20:
13         return "WARNING"
14     return "INFO"
15
16
17  def registrar_cambio(log, producto, antes, ahora):
18     """Registra con el nivel adecuado y devuelve el nivel."""
19     pct = variacion_pct(antes, ahora)
20     nivel = nivel_para(pct)
21     mensaje = f"{producto}: {antes:.2f} -> {ahora:.2f} ({pct:+.1f}%)"
22     getattr(log, nivel.lower())(mensaje)
23     return nivel

```

Por qué funciona: La decisión del nivel está separada del registro: nivel_para se puede probar sola, sin ningún logger. Y registrar_cambio acepta cualquier objeto con métodos info, warning, etc. (un logger real o uno falso): eso es lo que permite probarla.

9. APIs y HTTP · Hablar con la API de una tienda

Pista 1: `urlencode({"producto": "cable 2mm"})` da `producto=cable+2mm`.

Pista 2: Ordena las condiciones: primero los casos concretos (200, 404), después los rangos.

```

1  import json
2  from urllib.parse import urlencode
3
4
5  def construir_url(base, **params):
6      """URL con parámetros de consulta, sin los None."""
7      limpios = {k: v for k, v in params.items() if v is not None}
8      if not limpios:
9          return base
10     return f"{base}?{urlencode(limpios)}"
11
12
13 def procesar_respuesta(estado, cuerpo):
14     """Interpreta el código de estado y el cuerpo JSON."""
15     if estado == 200:
16         return json.loads(cuerpo)["productos"]
17     if estado == 404:
18         return []
19     if estado == 429 or estado >= 500:
20         raise ConnectionError(f"Error temporal {estado}: reintentar")
21     raise ValueError(f"Respuesta inesperada: {estado}")

```

Por qué funciona: Separar armar la URL e interpretar la respuesta de la petición HTTP en sí deja la lógica probable sin red. Clasificar los errores en temporales (`ConnectionError`) y definitivos (`ValueError`) es lo que permite que reintentar actúe solo cuando tiene sentido.

10. Empaquetado y distribución · Versionado semántico

Pista 1: Las tuplas se comparan elemento por elemento: `(1, 10, 0) > (1, 9, 3)`.

Pista 2: Desempaqueta las tres partes: `mayor, menor, parche = ...`

```

1  def comparar_versiones(a, b):
2      """
3      >>> comparar_versiones("1.10.0", "1.9.3")
4      1
5      """
6      ta = tuple(int(p) for p in a.split("."))
7      tb = tuple(int(p) for p in b.split("."))
8      if ta < tb:
9          return -1
10     if ta > tb:
11         return 1
12     return 0
13
14
15  def siguiente_version(version, parte):
16      """
17      >>> siguiente_version("1.4.2", "minor")
18      '1.5.0'
19      """
20     mayor, menor, parche = (int(p) for p in version.split("."))
21     if parte == "major":
22         return f"{mayor + 1}.0.0"
23     if parte == "minor":
24         return f"{mayor}.{menor + 1}.0"
25     if parte == "patch":
26         return f"{mayor}.{menor}.{parche + 1}"
27     raise ValueError(f"Parte desconocida: {parte}")

```

Por qué funciona: Convertir a tuplas de enteros delega la comparación en Python, que compara posición por posición. El versionado semántico comunica a quien usa tu paquete cuánto riesgo tiene actualizar.